BÀI TẬP CHƯƠNG 6

KIẾN TRÚC TẬP LỆNH

1. Chuyển đổi lệnh trong lập trình C thành lệnh hợp ngữ MIPS:

   a. f = -g + h + B[1];

Giải:

1. Giả sử kiểu dữ liệu của mảng B là int

Ta có:

```
1. sub  $t0, $s2, $s1      # $t0 = h - g
2. lw   $t1, 4($s7)        # $t1 = B[1] (offset = 1 * 4 bytes)
3. add  $s0, $t0, $t1      # f = -g + h + B[1]
```

   b. f = A[B[g] + 1];

Giải:

1. Giả sử kiểu dữ liệu của mảng A và B là int

Ta có:

```
2. sll  $t0, $s1, 2        # $t0 = g * 4 (biến g thành byte offset)
3. add  $t1, $s7, $t0      # $t1 = địa chỉ của B[g] = Base B + offset
4. lw   $t2, 0($t1)        # 0(thanh ghi) là lệnh lấy giá trị
5. addi $t3, $t2, 1        # $t3 = B[g] + 1 (đây là index)
6. sll  $t4, $t3, 2        # $t4 = index * 4 (biến index thành byte offset)
7. add  $t5, $s6, $t4
8. lw   $s0, 0($t5)        # A[B[g] + 1]
```

Biết rằng f, g, h, i, j được lưu lần lượt tại các thanh ghi $s0, $s1, $s2, $s3, $s4. Địa chỉ cơ sở của hai chuỗi A và B được lưu trong các thanh ghi $s6, $s7.

2. Hãy tìm câu lệnh C tương đương với các câu lệnh hợp ngữ MIPS sau:

a.

add $s0, $s0, $s1

add $s0, $s0, $s2

add $s0, $s0, $s3

add $s0, $s0, $s4

Giải:

Câu lệnh C: f = f + g + h + i + j;

b.

lw $s0, 4($s6)

Giải:

Câu lệnh C: f = *(A+1)// lệch 4 bytes tức là 1 index trong, và giải tham chiếu để lấy giá trị load lên.

3. Biểu diễn các lệnh sau dưới dạng mã máy:

a. add $t0, $t0, $zero

Giải:

B1: lệnh add thuộc R-type

B2: Op = 000000, shamt = 00000, funct = 100000

B3: rd = rs = $t0 = 8, rt = $zero = 0

Op	rs	rt	rd	shamt	funct
000000	01000	00000	01000	00000	100000

Binary: 0000 0001 0000 0000 0100 0000 0010 0000

Hexa: 0x01004020

b. lw $t1, 4($s3)

Giải:

B1: lệnh lw thuộc I-type

B2: Op = 100011

B3: rs = 19, rt = 9, immediate = 4

Op	rs	rt	immediate
100011	10011	01001	0000 0000 0000 0100

Binary: 1000 1110 0110 1001 0000 0000 0000 0100

Hex: 0x8E690004

c. and $t3, $s0, $s2

Giải:

B1: lệnh and thuộc R-type

B2: op = 000000, shamt = 00000, funct = 100100

B3: rs = $s0 = 16, rt = $s2 = 18, rd = $t3 = 11

Op	rs	rt	rd	shamt	funct
000000	10000	10010	01011	00000	100100

Binary: 0000 0010 0001 0010 0101 1000 0010 0100

Hex: 0x02125824

d. sll $t1, $t5, 7

Giải:

B1: lệnh sll thuộc R-type

B2: Op = 000000, shamt = 7, funct = 000000

B3: rs = 0 = 00000, rt = $t5 = 13 = 01101, rd = $t1 = 9 = 01001.

Op	rs	rt	rd	shamt	funct
000000	00000	01101	01001	00111	000000

Binary: 0000 0000 0000 1101 0100 1001 1100 0000

Hex: 0x000D49C0

NOTE: Cứ gặp lệnh dịch bit thì rs = 0.

e. addi $t0, $s3, 25

Giải:

B1: lệnh addi thuộc I-type

B2: Op = 001000, immediate = 25

B3: rs = $s3 = 19, rt = $t0 = 8

Op	rs	rt	immediate
001000	10011	01000	0000 0000 0001 1001

Binary: 0010 0010 0110 1000 0000 0000 0001 1001

Hex: 0x22680019

f. sw $t2, -48($s0)

Giải:

B1: lệnh addi thuộc I-type

B2: Op = 101011, immediate = -48

B3: rs = $s0 = 16, rt = $t2 = 10

Op	rs	rt	immediate
101011	10000	01010	1111 1111 1101 0000

Binary: 1010 1110 0000 1010 1111 1111 1101 0000

Hex: 0xAE0AFFD0

4. Chuyển các mã máy sau sang dạng hợp ngữ MIPS:

a. 1010 1110 0000 1011 0000 0000 0000 0100

Giải:

B1: Dựa vào Opcode (6 bit đầu + 2bit bên cùng bên trái chắc chắn là số 0 vì OP có mã hex lớn nhất cũng chỉ là 3x_hex) và xác định được là 0010 1011-> I-type và câu lệnh là sw

Op	rs	rt	immediate
101011	10000	01011	0000 0000 0000 0100

Op = 0010 1011 = 2B hex.

rs = 16 = $s0

rt = 11 = $t3

immediate = 4

Vậy mã máy này -> hợp ngữ MIPS là: sw $t3, 4($s0)

b. 1000 1101 0000 1000 0000 0000 0100 0000

Giải:

B1: Dựa vào 6 bit đầu và thêm 2 bit 0 bên cùng bên trái, ta xác định được OP là 0010 0011 -> 23 hex -> I-type và là lệnh lw.

Op	rs	rt	immediate
100011	01000	01000	0000 0000 0100 0000

Op = 23_hex.

rs = rt = 8 = $t0

immediate = 64

Vậy mã máy này -> hợp ngữ MIPS là: lw $t0, 64($t0)

Xác định các lệnh trên là thuộc kiểu lệnh gì? (I, R, J)

Giải:

Cả 2 câu lệnh ở câu a và b đều thuộc kiểu lệnh I-type.

5. Cho các mã máy như sau, hỏi tương ứng với từng mã máy là lệnh assembly gì của MIPS?

a. 0x01304024

Giải:

Binary: 0000 0001 0011 0000 0100 0000 0010 0100

B1: Lấy 6 bit đầu + 2 bit 0 bên cùng bên trái -> Op = 000000 -> R-type

Op	rs	rt	rd	shamt	funct
000000	01001	10000	01000	00000	100100

funct = 00 100100 = 24_hex -> Lệnh and

rs = 9 = $t1, rt = 16 = $s0, rd = 8 = $t0

Assembly: and $t0, $t1, $s0

b. 0x2128fff3

Giải:

Binary: 0010 0001 0010 1000 1111 1111 1111 0011

B1: Op = 00 001000 -> 08_hex -> I_type

Op	rs	rt	immediate
100011	01000	01000	1111 1111 1111 0011

rs = 01001 = 9 = $t1

rt = 01000 = 8 = $t0

immediate = 0xFFF3 -> là số âm -> Tìm độ lớn bằng cách bù 1 + 1:

0000 0000 0000 1100 = 0x000C + 1 = 0x000D = số 13 thập phân.

Assembly: addi $t0, $t1, -13

c. 0xad28fffc

Giải:

Binary: 1010 1101 0010 1000 1111 1111 1111 1100

B1: Op = 0010 1011 = 2B_hex -> I-type và là lệnh sw.

Op	rs	rt	immediate
101011	01001	01000	1111 1111 1111 1100

rs = 9 = $t1

rt = 8 = $t0

immediate: 0xFFFC là số âm, vì vậy lấy bù 2. Đảo bit: 0x0003 + 1 = 0x0004 -> số thức thời là -4

Assembly: sw $t0, -4($t1)

6. Cho giá trị của các thanh ghi sau: $t0 = 0x55555555, $t1 = 0x12345678. Hãy cho biết giá trị của thanh ghi $t2 sau khi chạy các lệnh sau:

a.

sll $t2, $t0, 4 *# $t2 = 0x55555550*

or $t2, $t2, $t1

Giải: $t2 = 0101 0111 0111 0101 0101 0111 0111 1000 = 0x57755778

b.

srl $t2, $t0, 3 #$t2 = 0x0AAAAAAA

andi $t2, $t2, 0xFFEF #vì là lệnh andi thì 0xFFEF = 0x0000FFEF, nếu là lệnh addi thì sẽ là 0x1111FFEF

Giải: $t2 = 0x0000AAAA

7. Cho đoạn lệnh assembly sau:

a.

and $t0, $s0, $s1 #t0 = 0x00000000

or $t1, $s0, $s1 #t1 = 0x1234567F

nor $t0, $t0, $t1 #t0 = 0xEDCBA980

sll $t0, $t0, 3 #t0 = 0x6E5D4C00

Biết trước khi chạy: $s0 = 0x12345678; $s1 = 0x00000007. Hỏi sau khi chạy xong đoạn lệnh trên, $s0, $s1, $t0, $t1 bằng bao nhiêu?

Giải:

s0 = 0x12345678, s1 = 0x00000007

t1 = 0x1234567F

t0 = 0x6E5D4C00

b.

andi $t0, $s0, 12 # t0 = 0x0000000C

nor $t0, $t0, $zero # t0 = 0xFFFFFFF3

ori $t0, $t0, 3 # t0 = 0xFFFFFFF3

srl $t0, $t0, 2 # t0 = 0x3FFFFFFC

Biết trước khi chạy: $s0 = 0x0000000f. Hỏi sau khi chạy xong đoạn lệnh trên, $s0, $t0 bằng bao nhiêu?

Giải:

s0 = 0x0000000F

t0 = 0x3FFFFFFC

8. Viết chương trình hợp ngữ tính tổng của các số nguyên tố nhỏ hơn 100.

Giải:

```
main:
    addi $t0, $0, 2
    add $t8, $0, $0

i_loop_start:
    slti $t1, $t0, 100
    beq $t1, $0, i_loop_end

    addi $t9, $0, 1
    addi $t2, $0, 2

j_loop_start:
    slt $t3, $t2, $t0
    beq $t3, $0, j_loop_end

    div $t0, $t2
    mfhi $t6
    beq $t6, $0, i_isnot_prime

    j j_increment

i_isnot_prime:
    add $t9, $0, $0
```

```
    j j_loop_end

j_increment:
    addi $t2, $t2, 1
    j j_loop_start

j_loop_end:
    beq $t9, $0, i_increment

sum:
    add $t8, $t8, $t0

i_increment:
    addi $t0, $t0, 1
    j i_loop_start

    i_loop_end:
```

9. Viết chương trình hợp ngữ nhập vào 3 số tự nhiên khác nhau, sau đó in ra 3 số này theo thứ tự giảm dần.

Giải:

```
    .text
    .globl main
    main:
        li $v0, 5
        syscall
        move $t0, $v0
```

```
    li $v0, 5
    syscall
    move $t1, $v0

    li $v0, 5
    syscall
    move $t2, $v0

    slt $t3, $t0, $t1
    beq $t3, $0, skip1
    move $t4, $t0
    move $t0, $t1
    move $t1, $t4
skip1:
    slt $t3, $t0, $t2
    beq $t3, $0, skip2
    move $t4, $t0
    move $t0, $t2
    move $t2, $t4
skip2:
    slt $t3, $t1, $t2
    beq $t3, $0, skip3
    move $t4, $t1
    move $t1, $t2
    move $t2, $t4
skip3:
    li $v0, 1
    move $a0, $t0
```

```
        syscall

        li $v0, 11
        li $a0, 32
        syscall

        li $v0, 1
        move $a0, $t1
        syscall

        li $v0, 11
        li $a0, 32
        syscall

        li $v0, 1
        move $a0, $t2
        syscall

        li $v0, 10
    syscall
```

10. Viết chương trình hợp ngữ cho phép nhập vào một ký tự in thường, sau đó in ra ký tự in hoa tương ứng với ký tự đã nhập vào.

Chữ thường và hoa cách nhau 32 bit trong mã ASCII, mà 32 bit tức là bit thứ 6 (2^5) của chúng khác nhau (1 cái 1 và 1 cái 0), còn lại đều giống nhau y chang. Vì vậy ta xử lý đơn giản là dùng phép AND để đưa chữ thường về chữ hoa.

Giải:

```
.text
.globl main
main:
    li $v0, 12          # Bật mã syscall 12: Đọc 1 ký tự từ bàn phím
    syscall
    move $t0, $v0       # Kéo luồng điện của ký tự vừa nhập cất vào $t0

    # dùng duy nhất 1 câu lệnh là andi $t0, $t0, ?. nhưng vậy số cần tìm ở đây là số mà không ảnh hưởng tới các bit khác (AND các bit khác vẫn giữ giá trị cũ) thì chắc chắn phải là số 1, vì AND số 1 không đổi giá trị. chỗ bit thứ 6 thì cần là số 0 để coi như là phép trừ cho số 32 vậy tức là ta sẽ andi với 1101 1111

    andi $t0, $t0, 0x000000DF

    move $a0, $t0       # Bơm kết quả từ $t0 sang thanh ghi đối số $a0
    li $v0, 11          # Bật mã syscall 11: In 1 ký tự ra màn hình
    syscall


    li $v0, 10          # Bật mã syscall 10: Exit an toàn
syscall
```